

MANAOKE · STUDY KIT RECIPE

Anki Decks & Printable Flashcards for Manaoke Songs

How the study kit is built — and how to make one for any song.

イノチミジカシコイセヨオトメ / Life is Short, Fall in Love,
Maiden

クリープハイプ · CreepHyp — 76 words, two ways to learn

Generated June 2026 · reproducible via `build_song_kit.py`

1 • What's in the kit

Two ways to learn every word in the song — one for your phone, one for your pocket.

The Anki deck .apkg

Every word as a spaced-repetition card with **embedded audio** — no internet needed once imported. Hear the Japanese, recall the meaning. 76 words → 132 cards.

The flashcard PDF .pdf

Print double-sided, cut, and study away from any screen. Japanese on the front, the answer on the back. For the later stage, when you already hear the song in your head and just want to drill the words — pocket-sized, no phone.

Both come from a single source — the song's own `data.json` and word list (and, for Anki, the same audio clips the website plays) — so the printed card and the Anki card never drift apart.

2 • Using the Anki deck

1. **Install Anki** (free desktop app, or AnkiMobile / AnkiDroid). Double-click the `.apkg`, or *File* → *Import*.
2. The deck appears nested under a parent **Manaoke** deck, named for the song — so multiple songs live side by side.
3. **Audio plays automatically** when a card is shown. The Japanese pronunciation is on the front; the English gloss is on the back.

Two card types

- **Recognize (JP → EN)** — see & hear the word, recall the meaning. Made for every word. This is the core skill for a song.
- **Produce (EN → JP)** — see the meaning, recall the word. Made only for content words (never for bare grammar particles). Roughly doubles your daily reviews.

Want listening practice only? In Anki's Browse window, select the 2 **Produce** cards and *right-click* → *Toggle Suspend*. Re-import later updates your cards in place — it never makes duplicates (see §6, stable GUIDs).

3 • Printing the flashcards

The one setting that matters: print double-sided with **“Flip on long edge.”** Get this right and every answer lands behind its own word. Get it wrong and the backs are scrambled.

Print dialog settings

SETTING	CHOOSE
Two-sided / Double-sided	On
Binding / Flip	Flip on long edge (macOS: Layout → Two-Sided → “Long-Edge binding”)
Scale	100% / Actual size — never “Fit to page”
Paper	US Letter, portrait

No auto-duplex printer? Print the odd pages (fronts), reload the stack, then print the even pages (backs). The same long-edge layout applies. Always do a one-sheet test and hold it to the light before printing all ten.

Then

- **Cut** along the faint grey grid lines — 8 cards per sheet, 3.5×2.5 inches each.
- The small number on each card lets you re-sort the deck if it gets shuffled.
- That's it — read the Japanese, recall the meaning, flip to check. No screen needed.

4 · Why the back page is mirrored

When you print on both sides and flip the sheet on its **long edge**, the paper turns left-to-right — like turning a page in a book. The top-left card becomes the top-*right*. So on the back page, each row's columns must be **reversed** for the answer to land behind its prompt.

FRONT PAGE		BACK PAGE (COLUMNS REVERSED)	
1	2	2	1
3	4	4	3
5	6	6	5
7	8	8	7

The card text itself is never flipped or rotated — the printer does the physical mirroring. The builder only changes *which cell each card sits in*. The page margins are kept equal left and right, which is what makes the mirror land exactly. (Short-edge flip would instead reverse the *rows* — the builder targets long-edge, the portrait default.)

5 · How it's built — the pipeline

One script, `build_song_kit.py`, does everything from the song's existing files:

```
# one command per song — outputs land in the chosen folder
python build_song_kit.py --song inochi --out ~/Downloads/Manaoke-Study-Kit
```

STEP	WHAT HAPPENS
1. Read	Parse the promoted song's <code>data.json</code> ; flatten all 7 sections into one ordered list of words (text, romaji, meaning, hint, lyric context, particle flag).
2. Resolve audio	For each word, find its clip via the canonical Manaoke slug <code>word_{section}_{rom_uid}</code> in <code>_assets/.../audio</code> — the same files the site plays.
3. Stage media	Transcode each Japanese <code>.wav</code> to <code>.mp3</code> (~10× smaller) with <code>ffmpeg</code> ; copy the English gloss mp3s. Dedupe by basename.
4. Anki	Build the <code>.apkg</code> with <code>genanki</code> : one shared note type, embedded <code>[sound:...]</code> audio, stable IDs & GUIDs.
5. PDF	Lay out 8 cards/sheet with mirrored backs and render with headless Chrome .

In the Anki deck, one basename ties the `[sound:]` tag to the bundled media file, so audio can never go missing. The printed cards carry no audio — that's deliberate (see §6).

6 · Design decisions & the things that bite

Drawn from how Anki and home-printer duplex actually behave. Each of these is a place an earlier, naïver version would have broken.

ANKI

- **Stable IDs, generated once.** The note-type ID and deck ID are hard-coded constants. If they changed between builds, Anki would treat every re-import as a brand-new deck and duplicate everything.
- **Stable per-word GUIDs.** Each note's GUID is derived from the *song* + *word position*, not from `genanki`'s default (a hash of all fields). That means editing a meaning or swapping audio later **updates** the existing card on re-import instead of creating a duplicate.
- **Audio is a `[sound:basename.mp3]` string in a field** — basename only, never a path. The file is shipped in the package's media list; only its basename is stored in-deck. Basenames must be unique & ASCII.
- **Re-include audio on the back template.** Anki does *not* auto-replay the front's audio via `FrontSide`, so the sound tag is placed on both sides.
- **mp3 over wav** (~10× smaller deck); **both light and dark mode** are styled (mobile uses `.nightMode`, desktop `.night_mode`); English fields are HTML-escaped because `genanki` writes raw HTML.

PRINTABLE CARDS

- **Long-edge flip → reverse columns** (§4). Equal left/right margins make it exact.

- **Pages are interleaved** front-1, back-1, front-2, back-2... so an auto-duplex printer pairs each physical sheet correctly. The final partial sheet is padded with blanks before mirroring so positions still line up.
- **Render with *headless* Chrome**, not the GUI “Save as PDF” (which ignores the CSS page size). The file URL must be **absolute** — `file:///./relative` silently loads nothing and yields a blank page.
- **Cut guides are *outlines*, and text is pure black on white** — both print without needing the “background graphics” flag, so the cards come out right on any printer.
- **No audio on the printout — by design.** Printed cards are for the later, screen-free stage of study, when the melody is already in your head. Audio belongs to the Anki deck; paper stays paper. (The front is left as a bare recall prompt: just the word.)

7 • Making a kit for the next song

Because everything derives from the song's own files, onboarding a new song is a few lines:

1. Add an entry to the `SONGS` table at the top of `build_song_kit.py`:

```
"silhouette": {
    # promoted version directory + its canonical audio assets
    "song_dir": f"{MANAOKE}/songs/silhouette-vNN",
    "assets": f"{MANAOKE}/songs/_assets/silhouette/audio",
    "base_slug": "silhouette",          # version-independent (stable deck)
    "live_slug": "silhouette-vNN",     # the slug that is actually live
},
```

2. Run it: `python build_song_kit.py --song silhouette --out ~/Downloads/Manaoke-Study-Kit`
3. Confirm the run prints the expected word/card counts and the PDF reports the expected page count.

One precondition: the song must have its per-word audio clips already generated under `_assets/.../audio` (the same clips the study cards on the site use). If a word is missing its JP clip, the builder stops and names it — generate it first, then re-run.

8 • Verifying a build

- **Anki:** unzip the `.apkg`; confirm note & card counts, and that every `[sound:]` reference has a matching media file (zero missing).
- **PDF:** `pdfinfo` reports US-Letter and $2 \times \text{sheet-count}$ pages. Rasterize the first front/back pair and eyeball the mirror (back row 1 reads 2 1).

- **Test print:** print one sheet double-sided, long-edge, and hold it to the light — each answer should sit directly behind its word.

Toolkit files: `build_song_kit.py`, `anki_card.css`, `print_card.css`. Requires `genanki`, `ffmpeg`, and Google Chrome. Built for Joe's Manaoke project — keep the durable copy in the project's `tools/` directory so every future song is one command away.